

Programming a 2D Ising Model/Accelerating the code

Contents hide

Beginning

■ 4. Accelerating the code

TASK 4a:

TASK 4b:

TASK 4c:

TASK 4d:

Page Discussion Read View source View history Tools ▀

This is the fourth section of the Ising model experiment.

Main Page: Programming a 2D Ising Model

Previous section: 3. Introduction to Monte Carlo simulation

Next section: 5. The effect of temperature

4. Accelerating the code

So far, your `energy()` and `magnetisation()` functions probably use Python loops to go over every spin in your system (and a double loop for the energy). One of the drawbacks of Python's flexibility is a performance penalty — though developing code in Python tends to be faster than other compiled programming languages, the resulting program is much, much slower.

Fortunately, the NumPy module provides an interface to a number of functions written to give high performance. These are written in the C programming language under the hood and take numpy arrays, which are contiguous arrays of a single data type, unlike Python lists, which may contain mixes of data types.

In this section, we are going to use these functions to improve the performance of our simulation.

When trying to optimise the performance of something, you should always measure first, rather than making assumptions. Although there are some very obvious candidates in our program, you should always measure properly.

The `ILtimetrial.py` script measures how fast it takes for 2000 iterations of your code to run.

The `ILProfiling.py` script is the same, except uses the python standard library package [cProfile](#) to sample what functions are being called and give you information about the time spent in each. Analysing the output of the script, you will see that by far the majority of the time is spent in the energy method, followed by the magnetisation one. This makes sense, as each Monte-Carlo step can call the energy function multiple times.

(If you want to profile your code in more detail, please seek advice from a demonstrator.)

In order to improve the performance of this, we need to:

- Run the energy call fewer times per step.
- Improve the performance of the energy function.

Fortunately, the NumPy module provides an interface to a number of functions written to give high performance. In this section, we are going to use these functions to improve the performance of our simulation.

TASK 4a:

Use the script `ILtimetrial.py` to record how long your *current* version of `IsingLattice.py` takes to perform 2000 Monte Carlo steps. This will vary, depending on what else the computer happens to be doing, so perform repeats and report the error in your average!

TASK 4b:

Look at the documentation for the NumPy `sum` [function](#). You should be able to modify your `magnetisation()` function so that it uses this to evaluate `M`. The energy is a little trickier. Familiarise yourself with the NumPy `roll` [and](#) `multiply` [functions](#), and use these to replace your energy double loop (you will need to call `roll` and `multiply` twice!). Add your faster code to the report.

TASK 4c:

Use the script `ILtimetrial.py` to record how long your *new* version of `IsingLattice.py` takes to perform 2000 Monte Carlo steps. This will vary, depending on what else the computer happens to be doing, so perform repeats and report the error in your average! What makes this new version faster?

TASK 4d:

Making the code even faster : Consider trying to make the Monte Carlo step function substantially faster by calculating the difference in energy directly, in a way that avoids looping over the whole lattice. Think back how you answered some of the introduction questions. You can make it so the energy and magnetisation only needs to be calculated explicitly at the beginning of the simulation.

This is the fourth section of the Ising model experiment.

Main Page: Programming a 2D Ising Model

Previous section: 3. Introduction to Monte Carlo simulation

Next section: 5. The effect of temperature

Appearance hide

Text

☒ Small

☐ Standard

☐ Large

Width

☒ Standard

☐ Wide